

Complex Systems — Normal Exam (Recreated)

2025/2026

Question	Points
1 – Define	18
2 – Comment	12
3 – Solve	20
4 – Explain	25
5 – Model	25
Total	100

Question 1 — Define

[18 points]

Define the following concepts in a clear and **rigorous** way. A concrete example may be used to help explain the concept.

- a) Bifurcation diagram
- b) Basin of attraction
- c) Diameter (of a graph)

Question 2 — Comment

[12 points]

Comment the following statements. Remember: to comment is not just to agree/disagree; you must **justify** your evaluation.

- a) A system described by differential equations that is non-autonomous and of order $n > 1$ can be converted into an autonomous system of order $n = 1$.
- b) There is a relationship between the Feigenbaum constant and the bifurcation diagram.

Question 3 — Solve

[20 points]

Consider a 1D cellular automaton with neighborhood of radius 1. Someone claims to have observed the following state transition:

$$\begin{array}{lcl} t : & 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0 \\ t + 1 : & 0, 1, 1, 0, 0, 1, 0, 1, 0, 1, 0 \end{array}$$

Is this situation possible? Justify your answer. What was the transition rule used and, if erroneous, correct it and present the new result.

Question 4 — Explain

[25 points]

In class we discussed the **Euler method** and the **Runge–Kutta (RK4)** method for numerically solving differential equations. Describe how each works and compare their advantages and disadvantages.

Question 5 — Model

[25 points]

You are building a **game-recommendation** platform with the following data:

- User-game interactions: (`user_id`, `game_id`, `rating`, `hours_played`)
- User metadata: (`age`, `region`, `preferred_platforms`)
- Game metadata: (`genre`, `developer`, `release_year`)

You want to build a recommender using a Graph Neural Network.

- a) Define the graph representation: what are the nodes, the edges, and their features?
- b) For this task, would you use a **GCN** or **GraphSAGE**? Justify.

Answer Key

Answer Key — Q1 (Define)

a) Bifurcation diagram. A plot of the *control parameter* of a system (x-axis, e.g. r in the logistic map) against the set of values the trajectory settles onto in the long term (y-axis, the attractor). It reveals the *period-doubling cascade* (a single line splits into 2, 4, 8, ... branches at the Feigenbaum rate $\delta \approx 4.669$) leading to a dense, chaotic smear, with self-similar *periodic windows* interrupting the chaos.

b) Basin of attraction. The set of all initial conditions (seeds) whose trajectories converge, in the long term, to a given attractor. Every attractor has its own basin; the basins partition the state space.

c) Diameter. The *maximum eccentricity* over all nodes, i.e. the longest shortest-path distance between any pair of nodes: $\text{diam}(G) = \max_{u,v} d(u,v)$, where $d(u,v)$ is the shortest-path distance. It measures the “worst-case” separation in the graph. (Radius = min eccentricity; eccentricity of $u = \max_v d(u,v)$.)

Answer Key — Q2 (Comment)

a) TRUE. Two standard reductions, applied together:

- *Order reduction:* introduce a new variable for each derivative. A k -th order ODE $y^{(k)} = g(y, y', \dots, y^{(k-1)}, t)$ becomes a system of k first-order ODEs (set $v_1 = y'$, $v_2 = y''$, ...).
- *Autonomization:* introduce a clock variable $\tau = t$ with $\frac{d\tau}{dt} = 1$, and replace the explicit t by τ . The system no longer depends on time explicitly.

Cost (worth noting): each reduction *increases the dimension*. This is why a *forced* (non-autonomous) 2nd-order oscillator becomes a 3D autonomous system — and hence *can* be chaotic, while an unforced 1D/2D one cannot.

b) TRUE. The Feigenbaum constant $\delta \approx 4.669$ *quantifies the geometry of the period-doubling cascade* visible in the bifurcation diagram: it is the limiting ratio of the lengths of consecutive intervals between successive period-doubling bifurcations,

$$\delta = \lim_{n \rightarrow \infty} \frac{r_n - r_{n-1}}{r_{n+1} - r_n} \approx 4.669.$$

So each “fork” in the diagram occurs $\sim 4.669\times$ closer to the previous one, and the doublings accumulate at a finite point (onset of chaos). δ is *universal* — the same for any 1D map with a smooth quadratic maximum.

Answer Key — Q3 (Solve)

Not possible — the transition is inconsistent.

For each interior cell (positions 2–10), read the neighborhood (left, self, right) at t and the

output at $t + 1$ (boundaries fixed):

pos	nbhd	out
2	010	1
3	100	1
4	001	0
5	010	0 (conflict!)
6	101	1
7	011	0
8	111	1
9	110	0
10	100	1 (agrees with pos 3)

The neighborhood **010** maps to **1** at position 2 but to **0** at position 5. A deterministic CA rule must send each neighborhood to a *single* output, so this observation is **impossible**.

Correction. Fix $010 \mapsto 1$ (first occurrence). Then position 5 must be 1, not 0. Corrected row:

$$t + 1 : \quad 0, 1, 1, 0, \mathbf{1}, 1, 0, 1, 0, 1, 0.$$

Extracted rule (outputs for 111, 110, 101, 100, 011, 010, 001, 000):

$$1, 0, 1, 1, 0, 1, 0, ?$$

The neighborhood **000** **never appears**, so the rule is *underdetermined*. If one assumes $000 \mapsto 0$, the binary string 10110100 gives **Wolfram rule 180**.

Answer Key — Q4 (Explain)

Setup. Both numerically integrate $\frac{dx}{dt} = f(x)$ from x_0 in steps of size h ; they differ only in how they estimate the slope across each step.

Euler method. Take the slope at the *current* point and walk a straight line along it:

$$x_{n+1} = x_n + h f(x_n).$$

Geometrically it approximates the curve by its tangent at the start of each step. The curve bends away, so errors accumulate. It is *first order*: global error $\propto h$.

Runge–Kutta (RK4). Sample the slope at *four* points across the step and take a weighted average:

$$\begin{aligned} k_1 &= f(x_n), & k_2 &= f\left(x_n + \frac{h}{2}k_1\right), & x_{n+1} &= x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4). \\ k_3 &= f\left(x_n + \frac{h}{2}k_2\right), & k_4 &= f(x_n + h k_3), \end{aligned}$$

By probing how the slope changes over the step (midpoints weighted double), it cancels most curvature error. It is *fourth order*: global error $\propto h^4$.

Comparison.

- *Accuracy*: RK4 error $\propto h^4$ vs Euler $\propto h$ — halving h cuts Euler's error in half but RK4's by $16\times$.
- *Step size*: RK4 stays accurate with much larger h , often needing fewer total steps despite more work per step.
- *Curvature/stability*: Euler (single start-slope) drifts and can diverge on curved/stiff problems; RK4 tracks the curve faithfully.

- *Cost/simplicity (Euler's only edge)*: Euler uses 1 evaluation per step and is trivial to implement; RK4 uses 4 evaluations per step and is more complex — but its accuracy gain almost always wins, so RK4 is the default.

One line: Euler = 1 slope (the start), $O(h)$, drifts; RK4 = 4 slopes averaged, $O(h^4)$, accurate.

Answer Key — Q5 (Model)

a) Representation — a bipartite / heterogeneous graph.

- **Nodes:** two types — *users* and *games*.
- **Edges:** a user–game edge exists when a user has interacted with a game (rated/played it). The graph is *bipartite* (edges only between a user and a game, never user–user or game–game).
- **Node features:**
 - User nodes: `age`, `region`, `preferred_platforms`.
 - Game nodes: `genre`, `developer`, `release_year`.
- **Edge features:** `rating`, `hours_played`.

Task framing: recommendation = *link prediction* (predict missing user–game edges) or *edge regression* (predict the rating a user would give a game). The head combines the two endpoint embeddings (dot product or concatenation → MLP) to score the (user, game) pair.

b) GraphSAGE. Reasons:

- **Inductive setting:** new users and new games appear constantly. A recommender must embed *unseen* nodes *without retraining*. GraphSAGE samples a fixed-size neighborhood and learns an *aggregation function*, so it generalizes to new nodes/graphs.
- **GCN is transductive:** it bakes in the full adjacency matrix, so a new node changes the matrix and forces retraining — impractical here.
- **Scalability:** GraphSAGE's neighborhood sampling scales to millions of users/games; full-graph GCN operations do not.

Conclusion: use **GraphSAGE** (inductive, scalable, handles the constant arrival of new users and games).